DeFi Fundamentals

By Code Eater

Contents

What is DeFi?

AMM

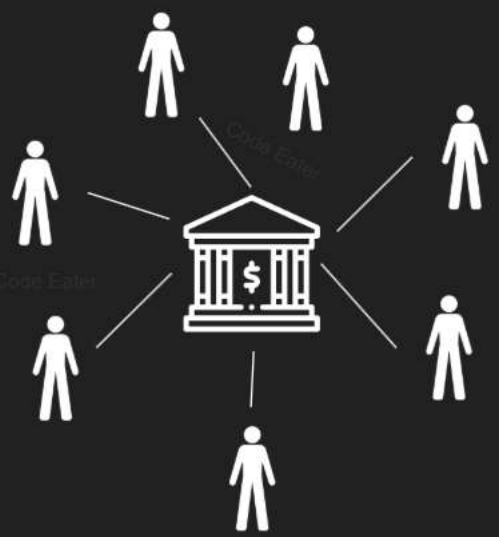
AMM workflow

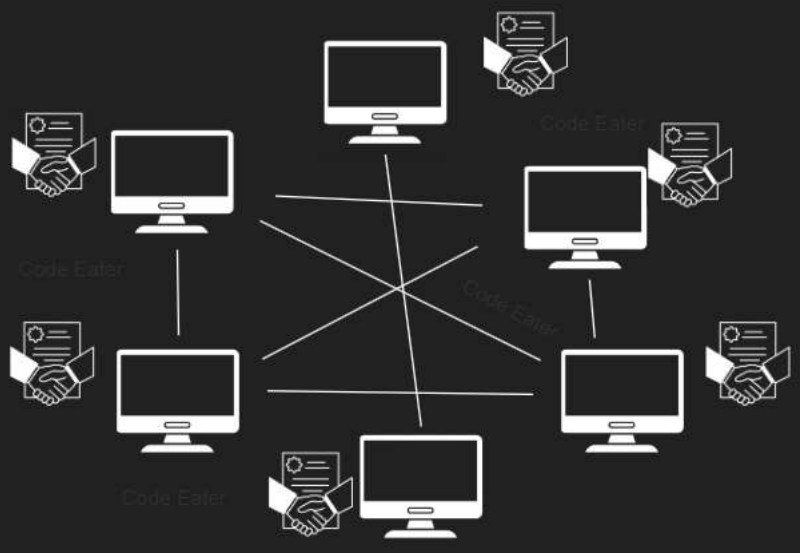
AMM architecture

Types of AMM

What is DeFi?

DeFi is **short for decentralized finance**, an umbrella term for peer-to-peer financial services on public blockchains.





Code Eater

DeFi Platforms

PancakeSwap
https://docs.pancakeswap.finance

PancakeSwap Intro - PancakeSwap

PancakeSwap is the leading decentralized exchange on BNB Smart Chain, ... **We're built on open-source software**: our site and all our Smart Contracts are ...

Wikipedia
https://en.wikipedia.org › wiki › Uniswap

Uniswap

It's **an open source project** and falls into the category of a DeFi product (Decentralized finance) because it uses smart contracts to facilitate trades. The ...

Platform: Ethereum License: GNU General Public License v3.0

Code Eater

DeFi Platforms

DeFi platforms

From sources across the web

AAVE	Uniswap	Curve Finance
Compound	PancakeSwap	Crypto.com
Decentralized exchange	Ethereum	Dai
Yearn Finance	AQRU	Avalanche
Binance	Defi lending and borrowing platform	DeFi Swap
Polkadot	SushiSwap	YouHodler
Balancer	GMX	Lending
Lido	Nexo	Solana

Show less

Feedback

Code Eater

CeFi Platforms

Binance	Coinbase	Gemini
Bitfinex	Huobi	OKX
Crypto.com	CEX.io	Bitstamp

DeFi Features

- Open
- Non-custodial
- Anonymous
- Zero Downtime

Code Eater

Automated Market Maker

AMM stands for Automated Market Maker. It's a decentralized finance (DeFi) concept and a key component of many decentralized exchanges (DEXs). Used in: Most decentralized exchanges (like Uniswap, SushiSwap, Balancer).

- **Automated** means that the process is carried out by computer programs or algorithms, without the need for manual intervention. Automated systems can execute tasks, make decisions, and respond to market conditions based on predefined rules or parameters.
- In finance, a **market** refers to a platform or environment where assets are bought and sold. It's a place where buyers and sellers come together to trade financial instruments like stocks, bonds, commodities, or in the case of cryptocurrency, digital assets.
- A **maker** is typically someone who provides liquidity to a liquidity pool. This involves adding their funds in the form of cryptocurrency pairs to the pool.

History Of AMM

2016

Vitalik Buterin

Proposed the implementation of on-chain AMMs.

2017

Bancor Protocol

Buterin's idea was first applied in the Bancor Protocol.This allowed trades between the Bancor Network Token (BNT) —the protocol token—and other ERC-20 tokens through specific smart tokens.

2018

Constant Product Formula

Uniswap v1 was launched with the implementation of the constant product formula and the liquidity pools ,and allowing trades between ETH and any other ERC-20 token.

2020

Curve Finance

Curve Finance launched a new AMM that was designed exclusively to trade between stablecoins, with low price impact and high efficiency

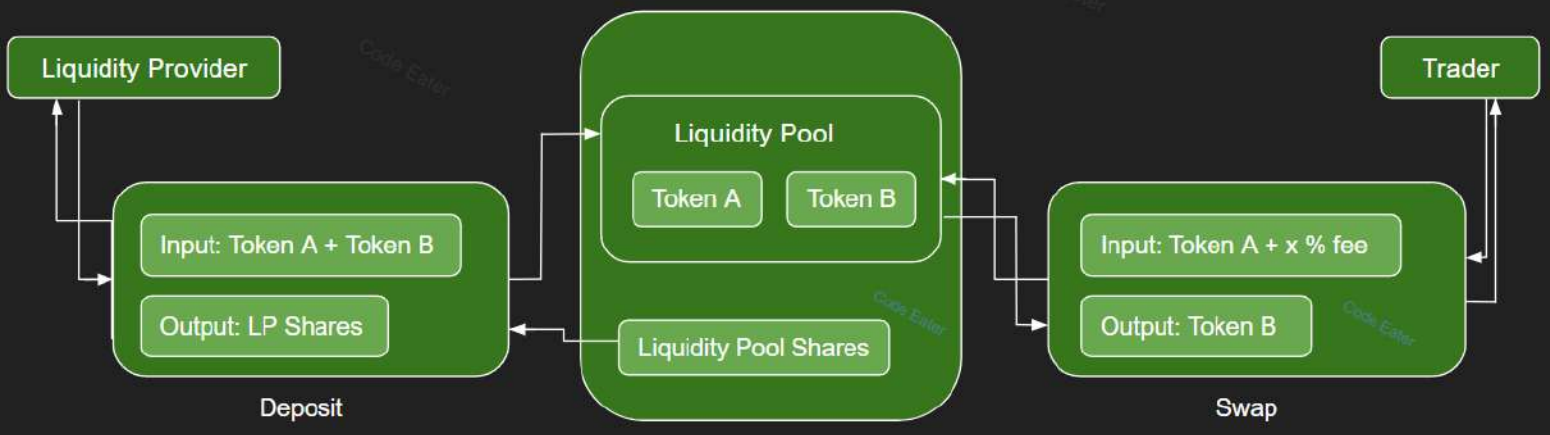
2021

Uniswap V3

Uniswap v3 appeared and started offering an entirely new concept to liquidity providers: concentrated liquidity.

Code Eater

AMM workflow



Liquidity

In the context of Decentralized Finance (DeFi), liquidity refers to the availability of assets in a market that can be quickly and easily bought or sold without causing a significant impact on the asset's price.

High liquidity in DeFi markets means that there are ample assets (cryptocurrencies or tokens) available for trading, which ensures that transactions can occur smoothly and efficiently.

Liquidity Pool

It is a pool of tokens (usually two different types) locked in a smart contract. These pools facilitate the trading of tokens without the need for an intermediary.

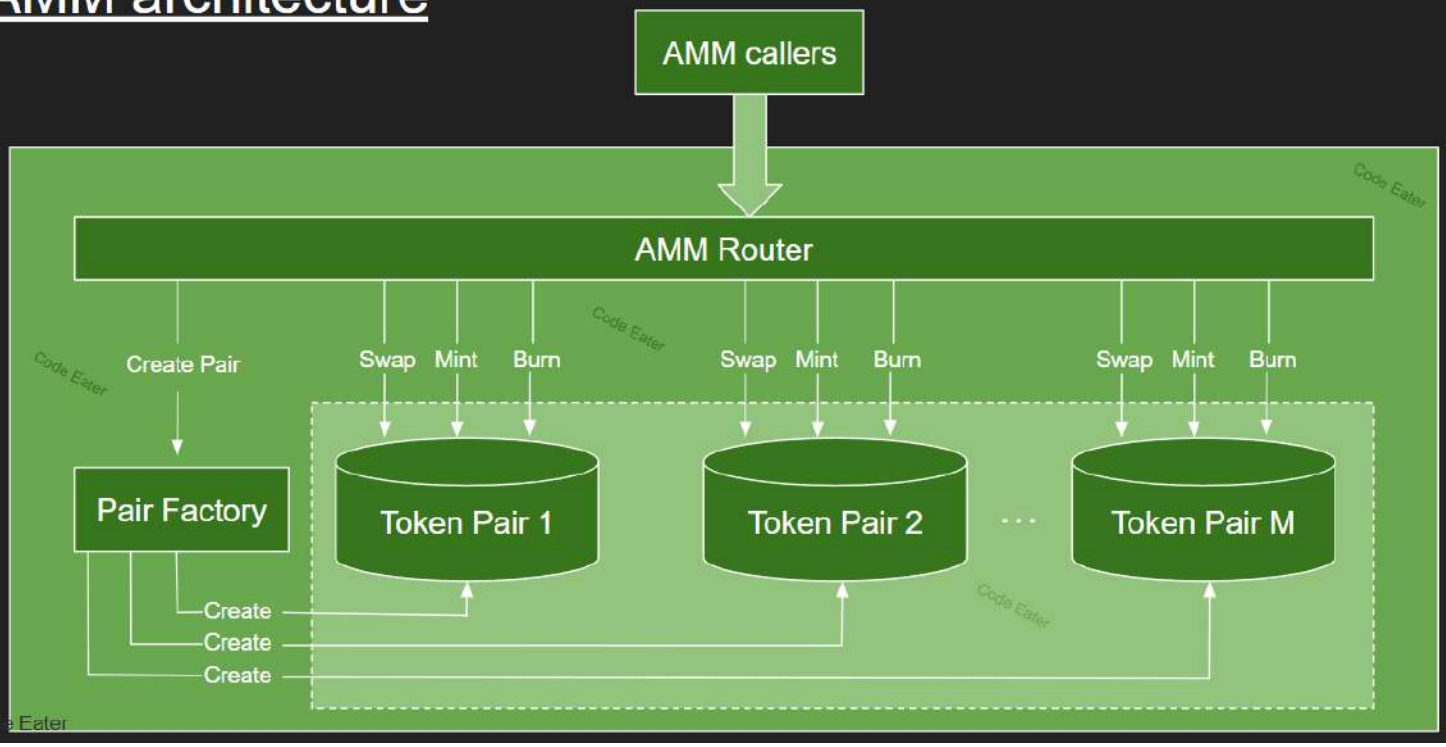
Generally, liquidity pools involve two tokens, often a popular cryptocurrency like Ethereum (ETH) and another token (e.g., a stablecoin like DAI or another cryptocurrency like LINK).The ratio of the tokens in the liquidity pool defines the prices of the two types of tokens in the pool.

The amount of liquidity in a liquidity pool determines the stability of the price when people are swapping tokens. When the liquidity pool is bigger, the ratio of the pooled tokens is more stable for a transaction of the same amount. People tend to trade using bigger liquidity pools to trade at a more stable price.

Order Book

In a DEX, order book operations are carried out through smart contracts on the blockchain. This means every action, such as placing an order, cancelling an order, or executing a trade, requires an on-chain transaction. And this will lead to high gas cost. Traditional centralized exchanges (like Binance, Coinbase) and some decentralized ones (like dYdX, 0x, Serum).
Example - Click Here

AMM architecture



AMM Callers

These are the **users or smart contracts** that interact with the AMM (e.g., DApps, wallets, or other protocols).

Code Eater
Code Eater
Code Eater
Code Eater

AMM Router

The AMM router is a smart contract that handles the interactions between users and various liquidity pools. It acts as an intermediary that finds the best trade routes across multiple pools to facilitate token swaps.

Routing Trades: When a user wants to swap tokens, the router finds the best path to complete the trade. This may involve multiple pools to get the best price or minimize slippage.

Optimizing Trades: The router optimizes the trade by splitting it across different pools if necessary, ensuring that users get the best possible rate.

Acts as the **interface** for users to:

- Swap tokens (e.g., Token A → Token B)
- `Mint` liquidity (add tokens to a pool and receive LP tokens)
- `Burn` liquidity (remove tokens from a pool by giving back LP tokens)

Internally, the router routes these actions to specific **Token Pairs**.

Code Eater

Pair Factory

A **pair factory** (often referred to simply as a "factory") is a smart contract responsible for creating and managing liquidity pools (pairs) on a decentralized exchange (DEX). It listens to "Create Pair" calls from the router or directly from callers.

User request	A user wants to create a new liquidity pool for a specific token pair (e.g., Token A and Token B).
Factory Interaction	The user interacts with the pair factory smart contract, specifying the two tokens.
Pool Creation	The factory contract checks if the pair already exists. If not, it deploys a new pair contract (liquidity pool) and initializes it.
Registration	The new pair's address is added to the factory's registry.
User Interaction	The user and others can now interact with the new liquidity pool, adding liquidity or trading tokens.

Code Eater

github.com/Uniswap/v2-periphery/blob/master/contracts/UniswapV2Router02.sol

Uniswap / v2-periphery

Type / to search

Code | Issues 49 | Pull requests 1 | Actions | Projects | Security | Insights

Files

master

Go to file

- .github
- buildV1
- contracts
  - examples
  - interfaces
  - libraries
  - test
  - UniswapV2Migrator.sol
  - UniswapV2Router01.sol
  - UniswapV2Router02.sol
- test
- .gitattributes
- .gitignore
- .mocharc.json
- .prettierrc
- .waffle.json
- .yarnrc
- LICENSE

v2-periphery / contracts / UniswapV2Router02.sol

NoahZinsmeister UniswapV2Router02 skeleton (#31) 6961711 · 5 years ago History

Code | Blame | 446 lines (420 loc) · 18 KB | Raw

```
pragma solidity =0.6.6;

import '@uniswap/v2-core/contracts/interfaces/IUniswapV2Factory.sol';
import '@uniswap/lib/contracts/libraries/TransferHelper.sol';

import './interfaces/IUniswapV2Router02.sol';
import './libraries/UniswapV2Library.sol';
import './libraries/SafeMath.sol';
import './interfaces/IERC20.sol';
import './interfaces/IWETH.sol';

contract UniswapV2Router02 is IUniswapV2Router02 {
    using SafeMath for uint;

    address public immutable override factory;
    address public immutable override WETH;

    modifier ensure(uint deadline) {
        require(deadline >= block.timestamp, 'UniswapV2Router: EXPIRED');
        _;
    }

    constructor(address _factory, address _WETH) public {
        factory = _factory;
        WETH = _WETH;
    }

```

Factory Contract

5. getPair

<input> (address)

0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2

<input> (address)

0x6B175474E89094C44Da98b954EedeAC495271d0F

Query

address

[getPair(address,address) method Response]

» address : 0xA478c2975Ab1Ea89e8196811F51A7B7Ade33eB11

Verify Here

`getPair(address tokenA, address tokenB)`

This function is part of the **UniswapV2Factory**, and it:

1) Takes two ERC-20 token addresses
2) Returns the address of the **liquidity pool (pair)** contract associated with them
3) If the pair doesn't exist, it returns `address(0)`

Uniswap Exchange- Click Here

etherscan.io/address/0x5c69bee701ef814a2b6a3edd4b1652cb9cc5aa6f#readContract

ETH Price: $3,815.44 (+3.60%) Gas: 0.74 Gwei

Search by Address / Txn Hash / Block / Token / Domain Name

Code | Read Contract | Write Contract

Connect to Web3

Read Contract Information

[Expand all] [Reset]

1. allPairs (0x1e3dd18b)

2. allPairsLength (0x574f2ba3)

3. feeTo (0x017e7e58)

4. feeToSetter (0x094b7415)

5. getPair (0xe6a43905)

<input> (address)

<input> (address)

<input> (address)

<input> (address)

Query

address

app.uniswap.org/explore/pools

Uniswap · Trade · Explore · Pool · Search tokens and pools · Connect

1D volume	Total Uniswap TVL	v2 TVL	v3 TVL	v4 TVL
$3.04B	$4.00B	$1.72B	$1.48B	$809.89M
▼ 9.65%	▼ 6.12%	▼ 7.54%	▼ 7.18%	▼ 0.83%

Tokens · Pools · Transactions · + Add liquidity · Protocol

#	Pool	Protocol	Fee tier	TVL	Pool APR	Reward APR	1D vol	30D vol	1D vol/TVL
1	WISE/ETH	v2	0.3%	$225.2M	0.01%	-	$24.9K	$375.3K	<0.01
2	ETH/USDC	v4	0.05%	$106.8M	19.2%	+7.11%	$112.3M	$5.4B	1.05
3	WBTC/USDC	v3	0.3%	$106.0M	11.69%	-	$11.3M	$540.0M	0.11
4	WBTC/ETH	v3	0.3%	$104.2M	3.11%	-	$3.0M	$120.6M	0.03
5	USDC/ETH	v3	0.05%	$84.7M	11.87%	-	$55.1M	$2.9B	0.65
6	ETH/USDT	v4	0.05%	$74.1M	7.36%	-	$29.9M	$1.6B	0.40
7	ETH/USDC	v3	0.05%	$73.8M	49.49%	-	$200.0M	$6.8B	2.71

Uniswap V2 Token Fetch

6. getAmountsOut (0xd06ca61f)

amountIn (uint256)

1000000

path (address[])

[0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48,0xdAC17F958D2ee523a2206206994597C13D831ec7]

Query

amounts *uint256[]*

[getAmountsOut(uint256,address[]) method Response]

» amounts *uint256[]* : 1000000,997825

Verify Here

amountIn (uint256)

- The amount of the input token you want to swap.
- For example, if you're swapping 1 DAI (18 decimals), you'd input `1000000000000000000`.

path (address[]

- The route the swap should follow.
- Example: `[DAI, WETH]` or `[DAI, WETH, USDC]`
- This determines which token pairs are used in sequence.

Types Of AMM

Constant Product => Uniswap

Constant Sum => Balancer

Curve-Based AMMs => Curve Finance

Hybrid AMMs => Sushiswap

Code Eater

Constant Product AMM

The CPMM is one of the most popular CFMM-based DEX. It means the product of the reserves of two or more tokens in a liquidity pool is a constant value:

$$\prod_{i=1}^{n} \text{Reserve}_i = K$$ where Reserve_i is the reserve of the i-th token, and K is a constant value.

Let say there are only two tokens (token A and token B) in a liquidity pool, we have the following:

$$\text{Reserve}_A \times \text{Reserve}_B = K$$

Constant Product AMM

Price of the tokens are determined by the equation: $XY = K$

X = Amount of Token A
Y = Amount of Token B

where K is the constant

Constant Product AMM

Number of X token = 100

Number of Y token = 200

Assuming Liquidity Pool Fee = 0

x*y = 20,000

Suppose a trader wants to buy 60 X tokens
In order to buy 60 X tokens trader needs to deposit Y tokens
After withdrawing 60 X tokens in the liquidity pool we will have 40 X tokens

40*y = 20,000

y = 20,000/40

y = 500

Number of Y tokens deposited by trader = 500 – 200 = 300

After deposit number of Y tokens = 300+200 = 500

x*y= 40*500 = 20,000

🧮 Given:

- Initial reserve of Token X = **50**
- Initial reserve of Token Y = **500**
- Liquidity pool fee = **0%**

📌 Task:

A trader wants to **buy 30 Token X**.

Price Of A Token In A Liquidity Pool

For the CPMMs that have two tokens in their reserve, the price of a token in a liquidity pool is the ratio of the reserve of the token and another token. Here, we suppose there are two tokens (token A and token B) in the liquidity pool.

$$\text{Price of Token A} = \frac{\text{Reserve}_B}{\text{Reserve}_A}$$

$$\text{Price of Token B} = \frac{\text{Reserve}_A}{\text{Reserve}_B}$$

Code Eater

Price Of A Token In A Liquidity Pool

Calculate the price of Token A in terms of Token B and vice versa.

Reserve of Token A = 100
Reserve of Token B = 200

$$\text{Price of Token A} = \frac{\text{Reserve}_B}{\text{Reserve}_A} = \frac{200}{100} = 2 \text{ Token B per Token A}$$

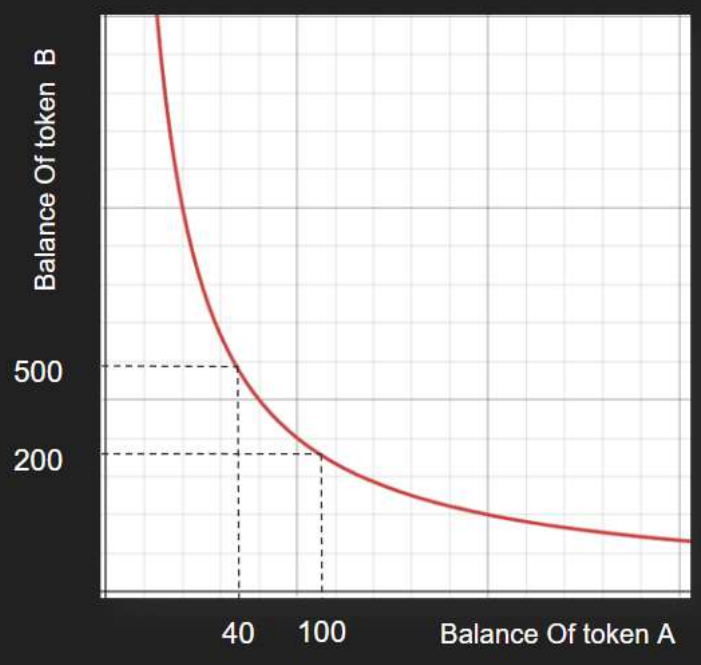
$$\text{Price of Token B} = \frac{\text{Reserve}_A}{\text{Reserve}_B} = \frac{100}{200} = 0.5 \text{ Token A per Token B}$$

Token A Price: If you want to buy 1 Token A, you need to provide 2 Tokens B.

Token B Price: If you want to buy 1 Token B, you need to provide 0.5 Tokens A.

Code Eater

Constant Product AMM Graph



Code Eater

AMM Terminologies and Mathematics

Contents

Constant Product AMM

Price Impact

TVL

Adding Liquidity

Minting Shares

Constant Product AMM Mathematics

Condition 1 - Trader is selling Token A and buying Token B, where Token A amount is X and Token B amount is Y initially in the pool.

dx = Amount of Token A in

dy = Amount of Token B out

Platform Fee = 0

Constant Product AMM Mathematics

How many tokens of B are out? [Hint: Find dy]

Before trade $XY = K$ — (1)

Before trade $(X+dx)(Y-dy) = K$ — (2)

$$(X+dx)(Y-dy) = K$$

$$X(Y - dy) + dx(Y - dy) = K$$

$$XY - Xdy + Ydx - dxdy = K$$

$$XY - dy(X + dx) + Ydx = XY \text{ (from eq 1)}$$

$$-dy(X + dx) + Ydx = 0$$

$$Ydx = dy(X + dx)$$

$$dy = \frac{Ydx}{X + dx}$$

Constant Product AMM Mathematics

How many tokens of A are out? [Hint: Find dx]

$$dy = \frac{Y\,dx}{X + dx} \qquad dx = \frac{X\,dy}{Y - dy}$$

Constant Product AMM Mathematics

Condition 2 - Trader is selling Token A and buying Token B, where Token A amount is X and Token B amount is Y initially in the pool.

dx = Amount of Token A in

dy = Amount of Token B out

Platform Fee = 0.3%

$$xy = k \quad - (1)$$

$$(x + dx(1 - Fee))(y - dy) = k \quad (2)$$

Trader → A → 1000 Tokens

Fee 0.3% of 1000

$$Fee = \frac{0.3}{100} \times 1000$$

$$Fee = 3$$

$$1000 - 3 = 997 \text{ toke}$$

Total Actual for trade =

$$\boxed{1000 - 0.3\% \text{ of } 1000}$$

$$1000(1 - 0.3\%)$$

$$1000(1 - Fee)$$

codeeater.pro is sharing your screen. Stop sharing Hide

3. **Solving for dy:**
   - Substitute the fee value of 0.3% (which is 0.003 as a decimal):

$$X \cdot Y = (X + dx \times 0.997) \cdot (Y - dy)$$

Expanding the product on the right-hand side:

$$X \cdot Y = X \cdot Y - X \cdot dy + dx \times 0.997 \cdot Y - dx \times 0.997 \cdot dy$$

Simplify to isolate dy:

$$0 = -X \cdot dy + dx \times 0.997 \cdot Y - dx \times 0.997 \cdot dy$$

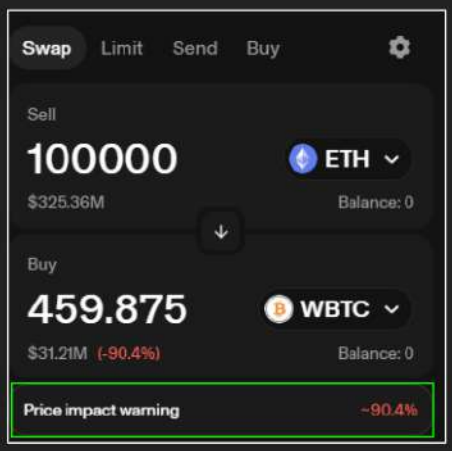
Factor out dy:

$$dy \times (X + dx \times 0.997) = dx \times 0.997 \cdot Y$$

Finally, solve for dy:

$$\boxed{dy = \frac{dx \times 0.997 \cdot Y}{X + dx \times 0.997}}$$

Price Impact



Here:

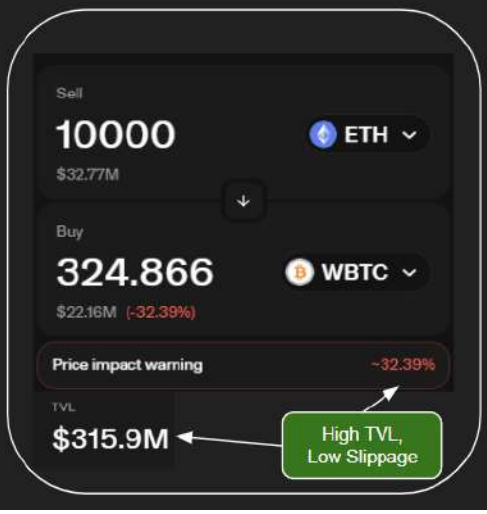
- 1 ETH ≈ 0.005 WBTC (just for example).
- So, 100,000 ETH **should give you** ≈ 500 WBTC normally.

But due to **price impact**, you're only getting **459.875 WBTC** — and even worse, the actual **value in USD is $31.21M**, while your ETH was worth **$325.36M**.

Price impact in a Constant Product Automated Market Maker (AMM) refers to the change in price resulting from a trade. When a trader buys or sells a token, it affects the reserves in the liquidity pool, and consequently, the price.

Price Impact & TVL

Higher TVL generally means deeper liquidity, which leads to lower price impact for trades



Total Value Locked

Total Value Locked (TVL) in DeFi refers to the total amount of assets (cryptocurrencies or tokens) that are currently being staked, locked, or otherwise used in decentralized finance protocols.

TVL is a key metric used to gauge the overall health, growth, and trust in the DeFi ecosystem. It indicates the amount of capital held within DeFi platforms and provides insight into the popularity and usage of these platforms.

For instance, if Uniswap has $5 billion worth of various tokens locked in its liquidity pools, that figure represents its TVL. This metric is a strong indicator of the platform's utilization and the trust users place in its services.

Total Value Locked

TVL is the sum of the value of both reserves in a liquidity pool. For a pool with Token A and Token B, the TVL is calculated as follows:

$$TVL = \text{Reserve A} \times \text{Price of Token A} + \text{Reserve B} \times \text{Price of Token B}$$

Price Impact Calculation

Reserve A =100

Reserve B = 200

K = 100 × 200 = 20 ,000

A trader buys 10 of Token A. Calculate the Price Impact.

Steps to Calculate Price Impact:

1. **Calculate Initial Price:**
   - The initial price of Token A in terms of Token B is:
   $$\text{Price}_{\text{initial}} = \frac{\text{Reserve}_B}{\text{Reserve}_A} = \frac{200}{100} = 2 \text{ Token B per Token A}$$
2. **Determine New Reserves After Trade:**
   - Since the trader buys 10 of Token A, the reserve of Token A decreases by 10:
   $$\text{Reserve}'_A = \text{Reserve}_A - 10 = 100 - 10 = 90$$
   - Using the constant product formula $K = \text{Reserve}'_A \times \text{Reserve}'_B$:
   $$\text{Reserve}'_B = \frac{K}{\text{Reserve}'_A} = \frac{20,000}{90} \approx 222.22$$

   This means the reserve of Token B increases.
3. **Calculate New Price:**
   - The new price of Token A in terms of Token B after the trade is:
   $$\text{Price}_{\text{new}} = \frac{\text{Reserve}'_B}{\text{Reserve}'_A} = \frac{222.22}{90} \approx 2.47 \text{ Token B per Token A}$$
4. **Calculate Price Impact:**
   - The price impact is the percentage change in price due to the trade:
   $$\text{Price Impact} = \left(\frac{\text{Price}_{\text{new}} - \text{Price}_{\text{initial}}}{\text{Price}_{\text{init}} \downarrow}\right) \times 100\% = \left(\frac{2.47 - 2}{2}\right) \times 100\% \approx 23$$

The price impact is approximately 23%, meaning the price of Token A in terms of Token B has increased by 23% due to the trade.

Price Impact & TVL

Scenario: A trader wants to buy 10 Token A from both pools.

Pool 1:

- Reserve of Token A: 1000
- Reserve of Token B: 2000

Pool 2:

- Reserve of Token A: 100
- Reserve of Token B: 200

Price Impact & TVL

Scenario: A trader wants to buy 10 Token A from both pools.

Pool 1:

- Reserve of Token A: 1000
- Reserve of Token B: 2000
- TVL: High

Pool 2:

- Reserve of Token A: 100
- Reserve of Token B: 200
- TVL: Low

Price Impact & TVL

Scenario: A trader wants to buy 10 Token A from both pools.

Pool 1 (High TVL):

- Initial Reserves: $\text{Reserve}_A = 1000$, $\text{Reserve}_B = 2000$
- New Reserves: $\text{Reserve}'_A = 990$, $\text{Reserve}'_B = \frac{1000 \times 2000}{990} \approx 2020.2$
- Initial Price: $\frac{2000}{1000} = 2$
- New Price: $\frac{2020.2}{990} \approx 2.04$
- Price Impact: $\left(\frac{2.04-2}{2}\right) \times 100\% \approx 2\%$

Higher TVL pools (deeper liquidity) have lower price impact because they can absorb larger trades with smaller changes in price

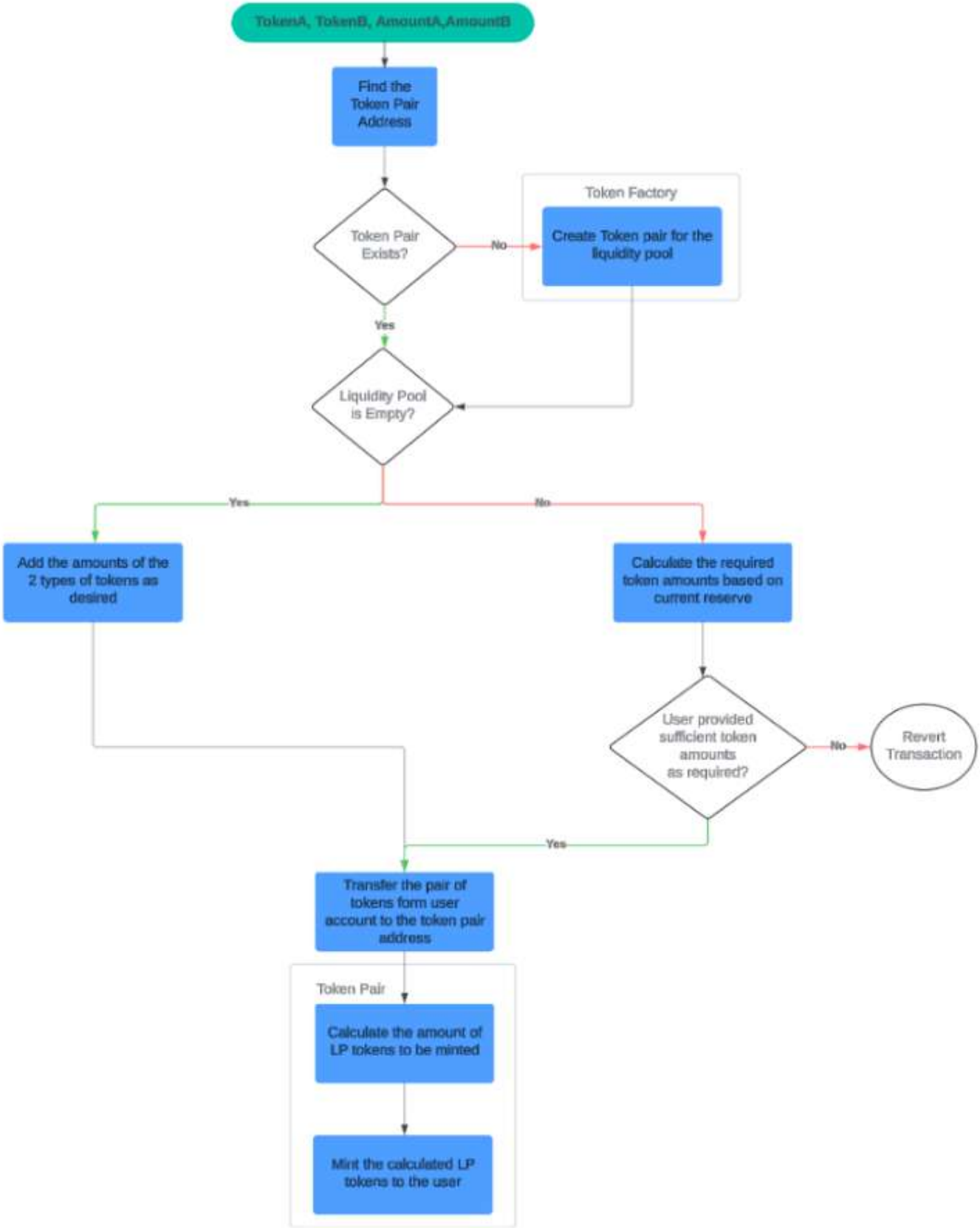
Pool 2 (Low TVL):

- Initial Reserves: $\text{Reserve}_A = 100$, $\text{Reserve}_B = 200$
- New Reserves: $\text{Reserve}'_A = 90$, $\text{Reserve}'_B = \frac{100 \times 200}{90} \approx 222.22$
- Initial Price: $\frac{200}{100} = 2$
- New Price: $\frac{222.22}{90} \approx 2.47$
- Price Impact: $\left(\frac{2.47-2}{2}\right) \times 100\% \approx 23.5\%$

Lower TVL pools (shallower liquidity) experience higher price impact from the same trade size due to more significant changes in the reserves' ratios.

Adding Liquidity

Adding Liquidity Flow Chart - Click Here


TokenA, TokenB, AmountA,AmountB
Find the Token Pair Address
Token Pair Exists?
No
Token Factory
Create Token pair for the liquidity pool
Yes
Liquidity Pool is Empty?
Yes
No
Add the amounts of the 2 types of tokens as desired
Calculate the required token amounts based on current reserve
User provided sufficient token amounts as required?
No
Revert Transaction
Yes
Transfer the pair of tokens form user account to the token pair address
Token Pair
Calculate the amount of LP tokens to be minted
Mint the calculated LP tokens to the user

🧪 Example:

You want to add liquidity to a new DEX for:

- **TokenA = USDC**
- **TokenB = ETH**
- You want to add: **AmountA = 1000 USDC, AmountB = 1 ETH**

✅ Step-by-Step Breakdown (with Reference to Flowchart):

---

1. Start with Inputs:

```ini
TokenA = USDC
TokenB = ETH
AmountA = 1000
AmountB = 1
```

---

2. Find the Token Pair Address:

- The system checks if a pool (pair) for `USDC-ETH` already exists.

---

3. Check: Does the Token Pair Exist?

- 🔷 If it **does not exist**:
  - Go to **Token Factory**.
  - **Create Token Pair for Liquidity Pool** → e.g., USDC-ETH pool is created.

---

4. Check: Is the Liquidity Pool Empty?

- If **Yes** (means it's the **first liquidity** being added):
  - ✅ You are allowed to **provide any ratio** of USDC and ETH.
  - So, the DEX **accepts 1000 USDC and 1 ETH** without validation.
  - ➡️ Then go to **Transfer the pair of tokens from user account to the token pair address.**

5. If the Liquidity Pool is NOT empty:

- This means there's already some liquidity in the pool.
- Now the DEX must **maintain the price ratio** of USDC:ETH.
  - Suppose current pool has:
    - 10,000 USDC and 10 ETH → 1 ETH = 1000 USDC
- So:
  - The system **calculates required ratio** for adding more liquidity.
    - You provide: 1000 USDC and 1 ETH → ✅ That's the correct ratio.
  - If the ratio is **wrong**, e.g. 500 USDC and 1 ETH → ❌
    - 👉 The system will **revert the transaction**.

---

6. Tokens Transferred to Pool:

- ✅ If the ratio is correct or it's the first liquidity:
  - The tokens are transferred from your wallet to the **pool address**.

---

7. LP Token Calculation and Minting:

- System calculates your share in the pool and gives you **LP tokens** (Liquidity Provider tokens).
  - These represent your **ownership share** in the pool.
  - E.g., if you contributed 10% of the total pool, you get 10% of LP tokens.
- LP tokens are **minted and sent to your wallet**.

Adding Liquidity

Suppose the pool has 100 units of token A and 200 units of token B. And there is an LP(Liquid Provider) who want to add some liquidity in the liquidity pool. What is the thing the LP should keep in mind?

Ration of Tokens = 100/200 = ½

In other words we can say for 1 token of A there are 2 tokens of B.

In short, this ratio is helpful to determine the price of a token.

Hence, it very important to maintain the ratio so that there is no price fluctuation.

And for that reason the LP while adding tokens to the liquidity pool should maintain the ratio of tokens.

Adding Liquidity

The ration in which the liquidity provider will add tokens will be -

$$\frac{x}{y} = \frac{dx}{dy}$$

Given:

- x and y are the initial amounts of token A and token B in the pool.
- dx and dy are the amounts of token A and token B being added to the pool.

Adding Liquidity

Suppose the pool has 100 units of token A and 200 units of token B. And there is an LP(Liquid Provider) who want to add some liquidity in the liquidity pool. What will be the amount of token A and token B that he should provide? Choose from below

Token A = 4 Token B = 16

Token A = 5000 Token B = 10000

Token A = 250 Token B = 800

Cannot Say

Adding Liquidity

Suppose the pool has 100 units of token A and 200 units of token B. And there is an LP(Liquid Provider) who want to add some liquidity in the liquidity pool. What will be the amount of token A and token B? Choose from below

Token A = 4 Token B = 16

Token A = 5000 Token B = 10000

It is maintaining the ratio of 1/2

Token A = 250 Token B = 800

Cannot Say

Adding Liquidity

What amount of dx and dy should be added in the liquidity keeping in mind that price of the tokens does not change? Initially the number of tokens are X and Y.

$$dx = x \cdot \frac{dy}{y}$$

$$dy = y \cdot \frac{dx}{x}$$

Liquidity Mining & Burning

Liquidity Mining - When providing liquidity, the liquidity pool takes tokens from the liquidity provider's wallet and mints new LP tokens for the liquidity provider, so we call the process of providing liquidity as liquidity mining.

Liquidity Burning - When removing liquidity from a liquidity pool, the liquidity pool takes the LP tokens from the liquidity provider's wallet and sends back the tokens from the reserve to the liquidity provider's wallet. Meanwhile, the LP tokens taken from the liquidity provider are burned, so we also call this process liquidity burning.

Minting Shares

The liquidity of a liquidity pool initially was L0. After sometime a liquidity provider added some liquidity to the liquidity pool due to which liquidity became L1. Assuming T is the total number of shares before adding the liquidity and s is the shares to mint. Find the value of s.

Given:

- L_0: Initial liquidity of the pool.
- L_1: New liquidity of the pool after adding liquidity.
- T: Total number of shares before adding liquidity.
- s: Shares to mint for the new liquidity provider.

Note - The proportion of the change in liquidity to the initial liquidity should be equal to the proportion of the new shares to the initial shares

Minting Shares

When a liquidity provider adds more liquidity, say ΔL, to the pool, the total liquidity becomes L_1:

$$L_1 = L_0 + \Delta L$$

$$\frac{\Delta L}{L_0} = \frac{s}{T}$$

Since the proportion of the change in liquidity to the initial liquidity should be equal to the proportion of the new shares to the initial shares

Solving for s:

$$s = T \times \frac{\Delta L}{L_0}$$

Since $\Delta L = L_1 - L_0$, we substitute ΔL with $L_1 - L_0$:

$$s = T \times \frac{L_1 - L_0}{L_0}$$

To be continued…

Measuring Liquidity

How to measure total liquidity from X and Y?

To measure the total liquidity in the pool in terms of tokens X and Y, we can use the geometric mean of the reserves:

$$\text{Total Liquidity} = \sqrt{x \cdot y}$$

Measuring Liquidity

A pool with reserves x=1,000 and y=1000. Calculate the liquidity.

To measure the total liquidity in the pool in terms of tokens X and Y, we can use the geometric mean of the reserves:

$$\text{Total Liquidity} = \sqrt{x \cdot y}$$

The geometric mean $\sqrt{1,000,000} = 1,000$

x.y vs sqrt(x.y)

Both x*y and sqrt(x*y) are valuable metrics, but they serve different purposes. x*y is fundamental for maintaining the AMM's pricing and balance mechanisms, while sqrt(x*y) provides a more intuitive and practical measure for comparing the liquidity across different pools.

Minting Shares

Step 1: Express L_0 and L_1

Given:

$$L_0 = \sqrt{xy}$$

$$L_1 = \sqrt{(x + dx)(y + dy)}$$

Minting Shares

When a liquidity provider adds more liquidity, say ΔL, to the pool, the total liquidity becomes L_1:

$$L_1 = L_0 + \Delta L$$

$$\frac{\Delta L}{L_0} = \frac{s}{T}$$

Since the proportion of the new liquidity to the initial liquidity should be equal to the proportion of the new shares to the initial shares

Solving for s:

$$s = T \times \frac{\Delta L}{L_0}$$

Since $\Delta L = L_1 - L_0$, we substitute ΔL with $L_1 - L_0$:

$$s = T \times \frac{L_1 - L_0}{L_0}$$

To be continued…

Previously…

(From Adding Liquidity Slide)

Step 2: Expand L_1

We know that:

$$dy = \frac{y}{x} \times dx$$

Thus, L_1 becomes:

$$L_1 = \sqrt{(x + dx)\left(y + \frac{y}{x}dx\right)}$$

Let's expand this expression inside the square root:

$$L_1 = \sqrt{(x + dx)\left(\frac{y(x + dx)}{x}\right)}$$

$$L_1 = \sqrt{(x + dx)\left(y + \frac{y \cdot dx}{x}\right)}$$

$$L_1 = \sqrt{(x + dx)\left(y + \frac{y \cdot dx}{x}\right)}$$

$$L_1 = \sqrt{xy + y \cdot dx + \frac{x \cdot y \cdot dx}{x} + \frac{y \cdot dx^2}{x}}$$

$$L_1 = \sqrt{xy + y \cdot dx + y \cdot dx + \frac{y \cdot dx^2}{x}}$$

$$L_1 = \sqrt{xy + 2y \cdot dx + \frac{y \cdot dx^2}{x}}$$

Step 3: Compute $L_1 - L_0$

Now, subtract $L_0 = \sqrt{xy}$ from L_1:

$$L_1 - L_0 = \sqrt{xy + 2y \cdot dx + \frac{y \cdot dx^2}{x}} - \sqrt{xy}$$

Step 4: Express s

The number of shares minted is given by:

$$s = T \times \frac{L_1 - L_0}{L_0}$$

Substituting the expression we found for $L_1 - L_0$:

$$s = T \times \frac{\sqrt{xy + 2y \cdot dx + \frac{y \cdot dx^2}{x}} - \sqrt{xy}}{\sqrt{xy}}$$

Step 5: Simplify the Expression

Let's simplify the expression under the square root:

$$L_1 - L_0 = \sqrt{xy\left(1 + \frac{2dx}{x} + \frac{dx^2}{x^2}\right)} - \sqrt{xy}$$

$$L_1 - L_0 = \sqrt{xy} \times \sqrt{1 + \frac{2dx}{x} + \frac{dx^2}{x^2}} - \sqrt{xy}$$

Thus:

$$s = T \times \frac{\sqrt{xy}\left(\sqrt{1 + \frac{2dx}{x} + \frac{dx^2}{x^2}} - 1\right)}{\sqrt{xy}}$$

$$s = T \times \left(\sqrt{1 + \frac{2dx}{x} + \frac{dx^2}{x^2}} - 1\right)$$

Therefore, the number of shares minted s is:

$$s = T \times \left(\sqrt{1 + \frac{2dx}{x} + \frac{dx^2}{x^2}} - 1 \right)$$

$$s = T \times \left(\sqrt{\left(1 + \frac{dx}{x}\right)^2} - 1 \right)$$

$$s = T \times \left(1 + \frac{dx}{x} - 1 \right)$$

$$s = T \times \frac{dx}{x}$$

🟢 PART 1: What Is a Liquidity Pool?

Before LP tokens, you must understand what a **liquidity pool** is.

🔹 Imagine:

You want to swap **USDC ⇄ ETH** on a decentralized exchange (DEX) like **Uniswap**.

- But there's no central authority matching buyers and sellers.
- Instead, there's a **pool of tokens** (USDC + ETH), and you trade **against this pool**.

That pool needs **liquidity** — and this comes from **users like you** who deposit both tokens into it.

🟢 PART 2: What Happens When You Add Liquidity?

When you add, say:

- 1 ETH
- 1,000 USDC

  to the pool, you're contributing to the **liquidity** others use for trading.

But how does the DEX track your deposit?

🔹 PART 3: LP Tokens — Your Proof of Deposit

> LP Token = A special ERC-20 token that **proves how much of the pool you own**.

- The smart contract mints LP tokens and sends them to your wallet.
- If you provided 10% of the pool's total value, you get 10% of the total LP tokens.

Example:

Pool Contents	LP Tokens in Circulation	You Deposit	Your LP Share	LP Tokens You Get
10 ETH + 10,000 USDC	1,000 LP Tokens	1 ETH + 1,000 USDC	10%	100 LP Tokens

Those **100 LP tokens** you receive:

- Represent **10% ownership** of the pool
- Earn you **10% of the trading fees**
- Can be **burned** to withdraw your tokens later

🟢 PART 4: LP Share

> LP Share = Your **percentage ownership** of the pool.

This is **not a token**, just a **concept** calculated using:

```text
LP Share = Your LP Tokens / Total LP Tokens
```

If:

- You have 100 LP tokens
- There are 1,000 LP tokens total

Then:

```text
LP Share = 100 / 1000 = 10%
```

🔹 PART 5: What Can You Do With LP Tokens?

Action	Effect
Hold LP Tokens	Continue earning a share of trading fees
Stake LP Tokens	Earn extra rewards in yield farms (like CAKE, UNI, etc.)
Transfer LP Tokens	Someone else now owns your share of the pool
Burn LP Tokens	Remove liquidity; get your tokens back based on your share of the pool

✅ Summary Table

Term	Meaning
Liquidity Pool	Pool of 2 tokens used for swapping on a DEX
Add Liquidity	Deposit both tokens in the correct ratio
LP Token	ERC-20 token proving your pool ownership
LP Share	Percentage of the pool you own (based on LP tokens)
Use of LP Token	Earn fees, withdraw tokens, stake, transfer, etc.
Burn LP Token	Remove liquidity and get your share of the pool back

🔥 What Does "Burn LP Tokens" Mean?

> **Burning LP tokens** = You return your LP tokens to the pool smart contract and **withdraw your share** of the liquidity.

- LP tokens are **destroyed** (burned).
- The pool sends you back your **portion of the two tokens** (e.g., ETH and USDC).
- Your LP share is **removed**.

📦 Example: Let's Break It Down

🏦 Pool State:

- **Total Pool Tokens**:
  - 100 ETH
  - 100,000 USDC
- **Total LP Tokens:** 1,000 LP

📦 Example: Let's Break It Down

🏦 Pool State:

- **Total Pool Tokens:**
  - 100 ETH
  - 100,000 USDC
- **Total LP Tokens:** 1,000 LP

👤 You:

- You have **100 LP tokens** (which is **10% of the pool**)

You decide to **remove your liquidity** → you burn your 100 LP tokens.

🔁 What Happens Internally:

1. The smart contract checks your LP share:

```text
LP Share = 100 / 1,000 = 10%
```

2. You receive:
   - **10% of ETH**: 100 ETH × 10% = **10 ETH**
   - **10% of USDC**: 100,000 USDC × 10% = **10,000 USDC**
3. Your 100 LP tokens are **burned** (removed from total supply).
4. Pool now contains:
   - **90 ETH**
   - **90,000 USDC**
   - **900 LP tokens remaining**

🧠 Key Points to Remember:

🔹 Action	🔸 Result
Burn LP Token	Withdraw your share of both tokens from the pool
Tokens Returned	Based on your LP share
LP Token Burned	Permanently removed from circulation
Pool Size Decreases	Because you're taking tokens out
No penalty	You get your share + fees earned while you held LP tokens

fuzzing is stateless fuzzing and invariants are stateful fuzzing
Foundry Invariant = Stateful fuzzing
Speed
Quality
Captions

4️⃣ Pool ke andar kya hota hai?

Socho pool ek dukaan hai:

Pehle:

- ETH = kam
- WBTC = zyada

  👉 ETH mehenga

Tumne kya kiya?

- 100,000 ETH pool me daal diya

  👉 ETH **flood** ho gaya (supply ↑↑)

Result:

- ETH ki value gir gayi 📉
- WBTC mehenga ho gaya 📈

Isliye:

```
500 WBTC ❌
459.875 WBTC ✅
```

5️⃣ Demand–Supply yahan kaise connected hai?

Tum kya karte ho	Pool kya feel karta hai	Price pe effect
ETH bechte ho	ETH supply ↑	ETH price ↓
WBTC kharidte ho	WBTC supply ↓	WBTC price ↑

👉 **Demand & supply yahin se banti hai**

6️⃣ Price Impact ka real meaning

Price Impact =

> Tumhari trade ki wajah se price kitni zyada badal gayi

- Choti trade → chota impact
- **Badi trade → bada impact** ❌

Is image me:

```java
Price Impact = -90.4%
```

👉 Matlab:

> Tumhari trade ne market ko hi hila diya 💥

🧠 Pehle TVL kya hota hai?

TVL (Total Value Locked) =

> Pool ke andar **total paisa** (ETH + token) kitna pada hai (USD me)

Example:

- ETH–WBTC pool = **$315.9M TVL**
- ETH–PMPY pool = **$2.1M TVL**

🔗 Golden Rule (yaad rakhna)

> **TVL zyada → liquidity deep → price impact kam**
> **TVL kam → liquidity shallow → price impact zyada**

LEFT SIDE (High TVL Pool) ✅

ETH → WBTC

- TVL = $315.9M 💰💰💰
- Tum sell kar rahe ho: **10,000 ETH**
- Price impact ≈ -32%

Kyun?

Socho pool ek **bahut badi nadi** hai 🌊

- Tum 1 balti paani daalo
- Water level thoda hi change hoga

👉 ETH zyada hone ke baad bhi pool disturb nahi hota
👉 Isliye **slippage kam**

RIGHT SIDE (Low TVL Pool) ❌

ETH → PMPY

- TVL = $2.1M 💧
- Same sell: **10,000 ETH**
- Price impact ≈ -97% 😱

Kyun?

Socho pool ek **chhota talaab** hai 🪣

- Tum tanker bhar paani daal do
- Talaab overflow / balance bigad gaya

👉 ETH supply achanak bahut zyada
👉 PMPY almost khatam
👉 Price crash